СОДЕРЖАНИЕ

Диаграмма последовательности (*sequence diagram*) 2
Описание элементов диаграммы последовательности 3
Уточнение диаграммы последовательности 9
Многоуровневая система диаграммы последовательности. 13
Практическая работа №4. 14

Диаграмма последовательности (sequence diagram)

Второй пример поведенческих диаграмм – это диаграмма последовательности.

Диаграмма последовательности – это UML-диаграмма, на которой для некоторого набора объектов на единой временной оси показан жизненный цикл объекта* (создание-деятельность-уничтожение некой сущности) и взаимодействие актеров (действующих лиц) информационной системы в рамках **прецедента**.

Примечание*: Диаграмма последовательности описывает систему, напрямую связанную с временным параметром. То есть, действия отслеживаются по времени. Это главное отличие последовательности от прецедентов.

Описание элементов диаграммы последовательности

При создании диаграммы последовательности стоит начать с определения действующих лиц, которыми снова могут выступать как одушевленные, так и неодушевленные объекты. Однако, теперь существует возможность разделить роли субъектов и объектов. (Ранее, мы говорили, что актер – это все сущности, в диаграмме последовательности же **объекты** и **субъекты** имеют разные модели).

Начнем, пожалуй, с субъекта, побуждающего к действию: в данной диаграмме, у всех субъектов существует ***собственный жизненный цикл*** и ***имя***. Жизненный цикл сам по себе – это пунктирная линия под актером. Существует она для того, чтобы показать «*полную жизнь*» человечка.

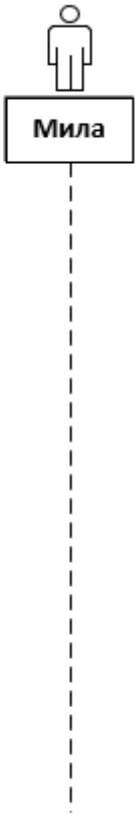


Рисунок 1 – Жизненный цикл субъекта.

Если у нас существует субъект, значит может существовать и объект – неодушевленный предмет.



Рисунок 2 – Жизненный цикл объекта.

Итак, мы имеем два жизненных цикла: один – субъекта, второй – объекта. В какой-то момент времени из их существования они будут взаимодействовать друг с другом (иначе в диаграмме нет смысла), чтобы это отобразить используют «**временной интервал**» или «**активация**». Необходим он для того, чтобы отобразить время взаимодействия двух сущностей*.

Если временной интервал не будет отображен, это означает, что две сущности взаимодействуют друг с другом от момента создания (рождения) до полного выхода из эксплуатации. Это в свою очередь противоречит самой сути ИС.

Примечание*: Сущность не может иметь время взаимодействия в начале и самом конце жизненного пути. Иначе (по нашей системе), это бы означало что Мила «родилась» с мячом или «умерла» от него. Поэтому всегда оставляйте «хвостик» жизненной линии у сущностей.

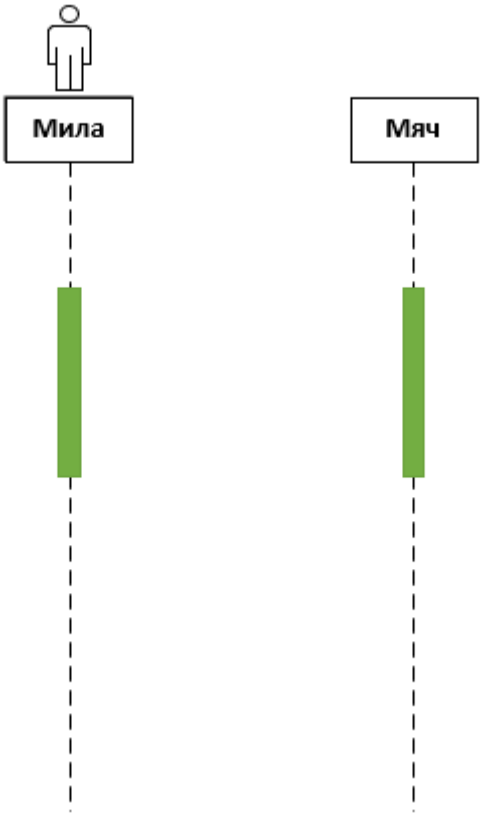


Рисунок 3 – Правильное выставление временного интервала.

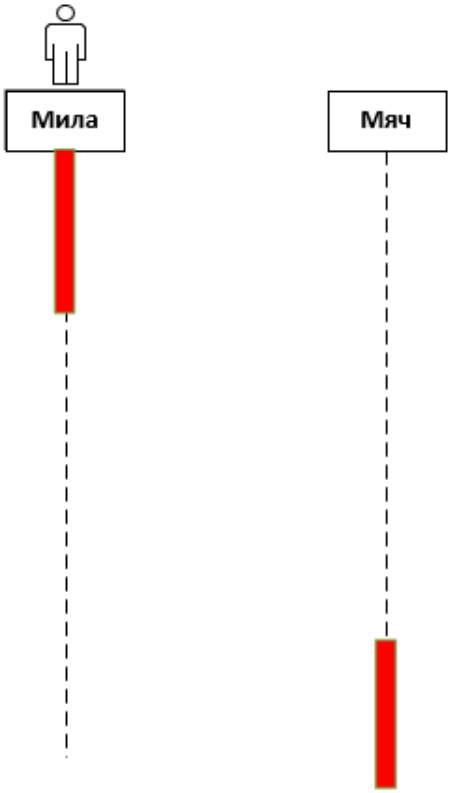


Рисунок 4 – Неправильное выставление временного интервала.

Когда время взаимодействия у нас определено верно и две сущности работают друг с другом *параллельно*, мы можем начать передавать между ними сообщения. Однако, сообщения тоже имеют разный вид. Начнем с «**отправочного сообщения**» или «**запроса**».

Запрос – это сообщение, отправляемое самым первым. Это первоначальная просьба о каком-то результате.

Например, пользователь нажимает на кнопку и видит всплывающее окно. В данном случае запросом будет являться сообщение «Нажать на» *.

Примечание*: если мы отправляем запрос или возвращаем результат, то его смысл формулируется глаголами без использования субъекта или объекта действия (Но могут использоваться союзы и предлоги). Например: «Нажать на», «Выйти из», «Вернуть», «Очистить», «Просмотреть» и т.д.

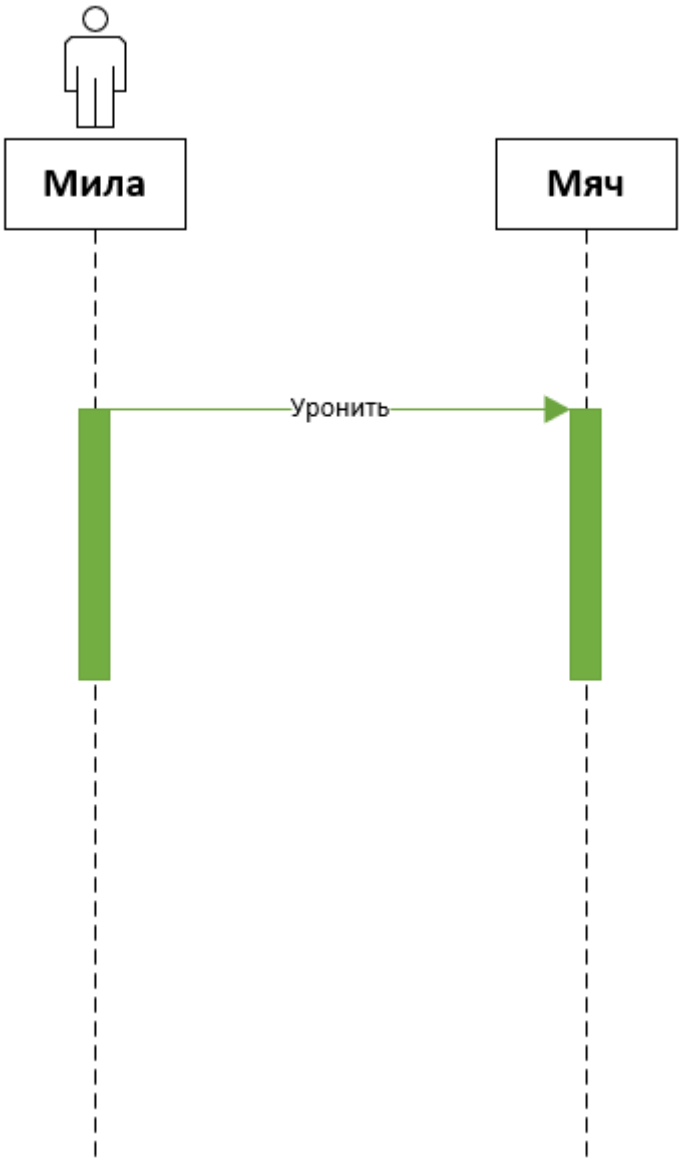


Рисунок 5 – Сообщение с запросом.

В изучаемой модели Мила – это источник запроса; субъект, который должен получить результат. Итогом любого запроса (первоначального сообщения) является «**возвращаемое сообщение**». Этот элемент выглядит как пунктирная линия со стрелкой. Возвращаемое сообщение – это следствие запроса.

Что касается смысловой нагрузки диаграммы последовательности – она может нарушаться. Запрос и результат – это не всегда одно и то же. Пользователь нажмет на кнопку, но в результате получит открытие формы. Запрос – «Нажать на», вывод – «Открытие формы». Данная структура

раскрывает последовательность действий, а не ожидаемые и фактические результаты.

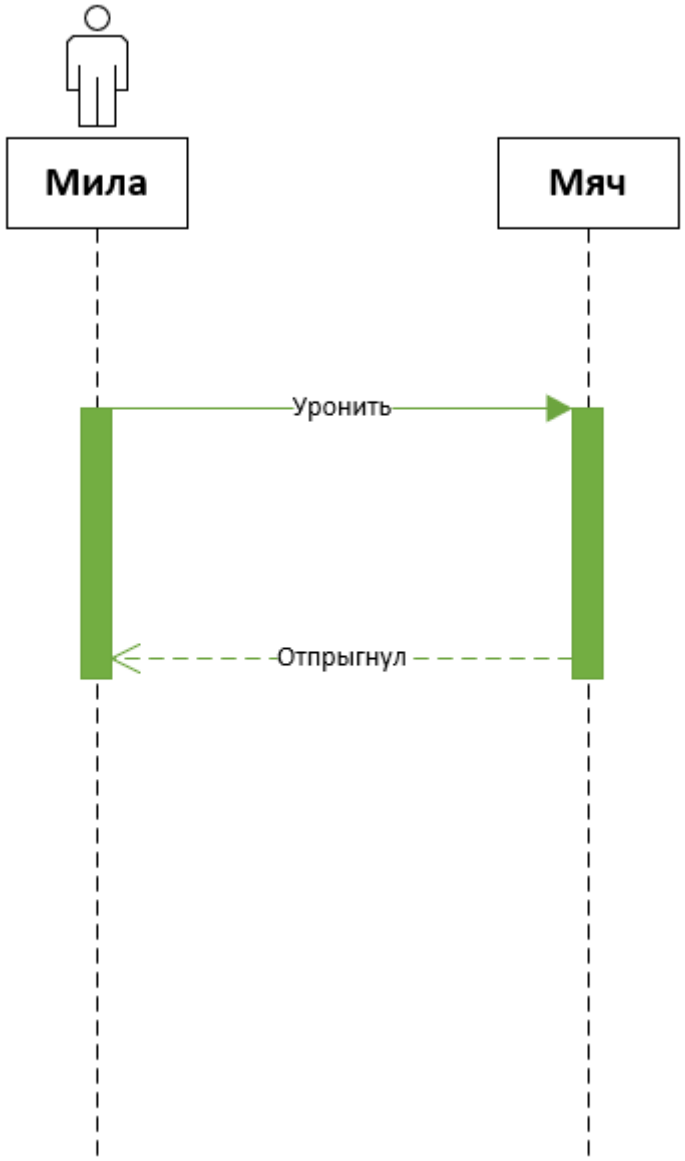


Рисунок 6 – Возвращаемое сообщение

Уточнение диаграммы последовательности

Данная диаграмма предназначена для того, чтобы мы могли определить программный переход данных и определить для самих себя необходимость в распределении привилегий, добавлении новых классов и функций…

Именно поэтому данная диаграмма может порой получиться очень развернутой. В этом случае могут добавляться новые элементы построения диаграммы.

Начнем с «**Асинхронного сообщения**» - это сообщение, которое передается дальше, исходя от основного запроса. Однако, асинхронное сообщение не несет в себе вывода.

В качестве примера возьмем авторизацию на сайте. Пользователь вводит свои данные на форму, которые должны проверяться в БД.

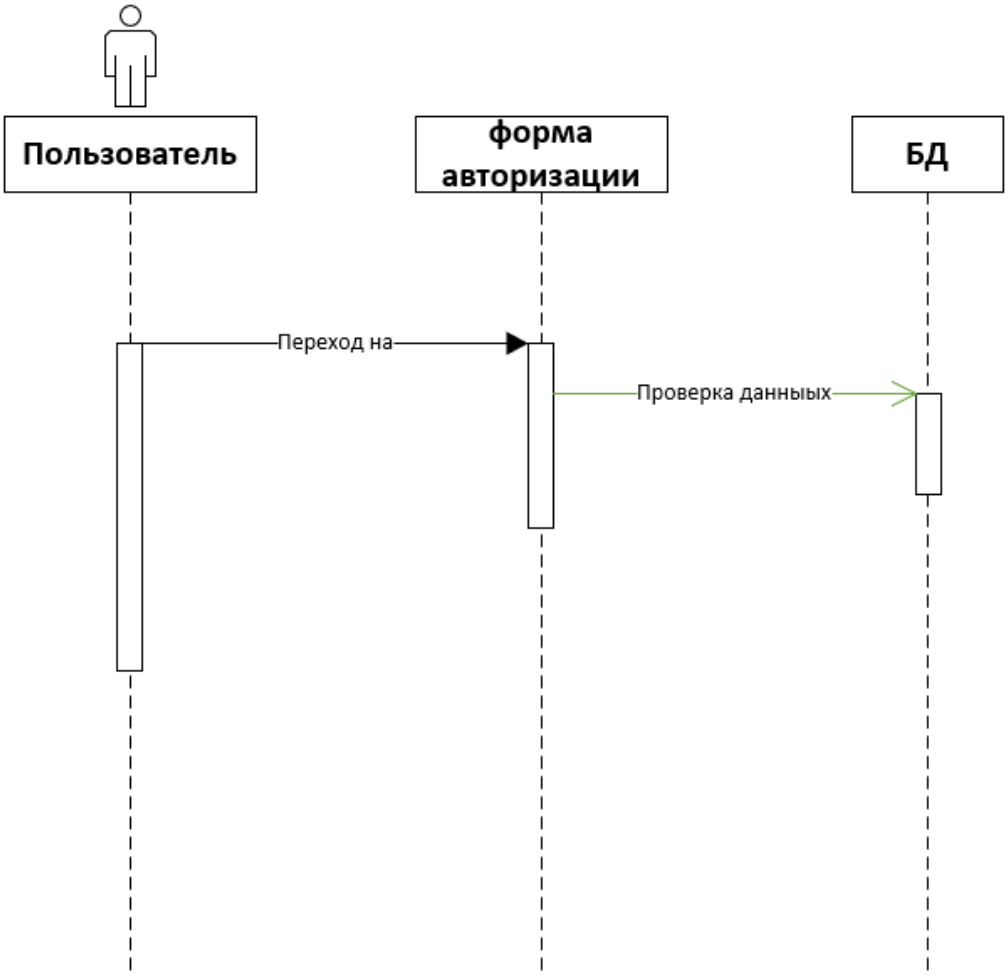


Рисунок 7 – Пример асинхронного сообщения.

Внесенные данные должны проверяться внутри БД. Значит, она будет обращаться сама к себе. Для отображения этого в диаграмме используется «**Самоосообщение**»*.

Примечание*: Сообщение самому себе может не иметь подписи. По умолчанию будет подразумеваться предыдущий поток сообщений.

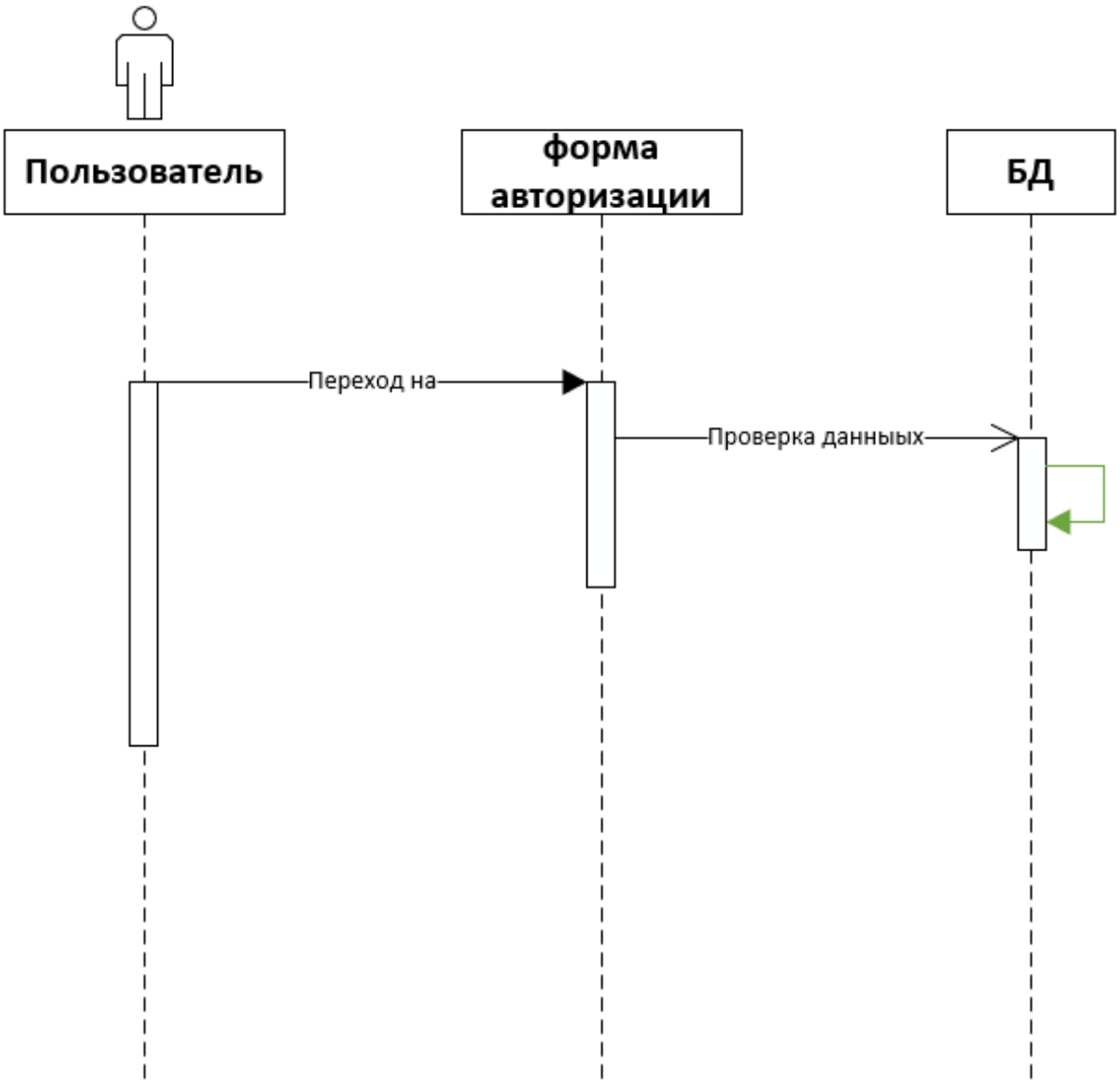


Рисунок 8 – Сообщение самому себе.

Что касается возвращаемых сообщений – здесь разницы не будет. Они будут исходить из всех сущностей, к которым дошел запрос, вплоть до конечного вывода.

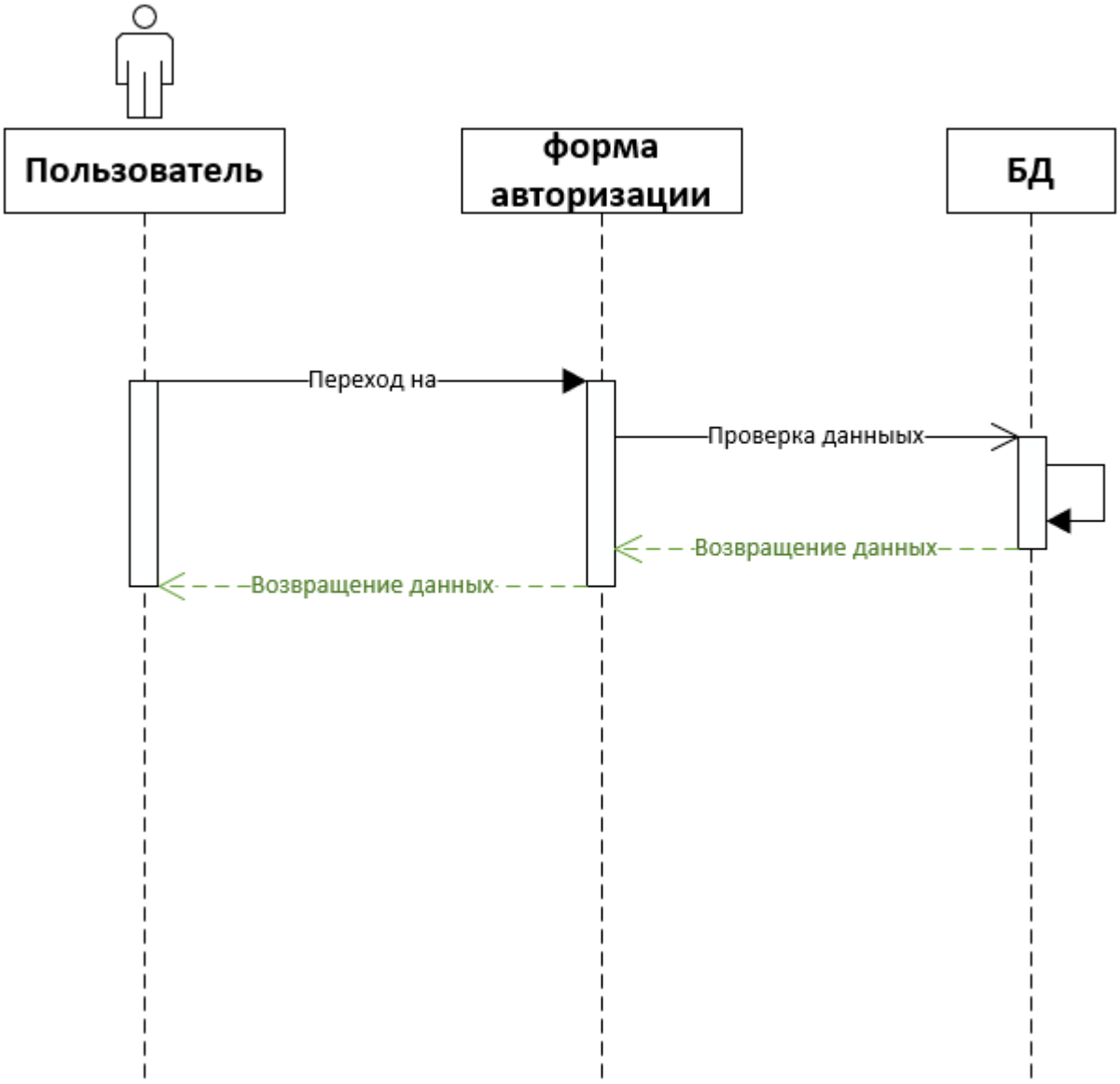


Рисунок 9 – Возвращаемое сообщение в сложной структуре.

А теперь вернемся к тому, что при работе с некоторыми элементами их жизненный цикл все же может закончится. Например, если взять работу с БД, то администратор при работе с определенной записью может ее удалить. Удаление – это и есть уничтожение, а чтобы показать это на диаграмме используют элемент «терминатор». **Терминатор** (в рамках диаграммы последовательности) – это графическое отображение окончания жизненного цикла объекта или субъекта.

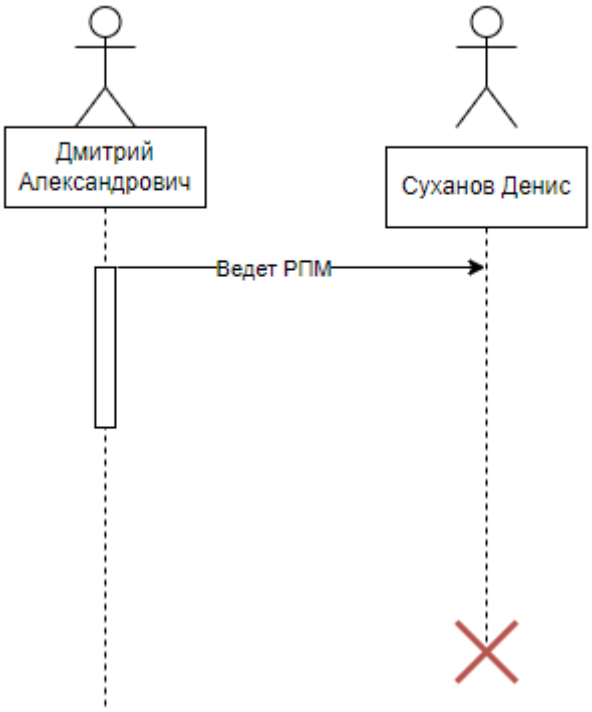


Рисунок 10 – Пример использования терминатора.

Примечание*: не всегда, но есть возможность того, что результатом на запрос будет являться как раз уничтожение объекта или субъекта.

Многоуровневая система диаграммы последовательности.

В иных случаях может использоваться несколько запросов. Например, в программе будет несколько кнопок или форм. Значит запросов на работу с ними будет несколько. Обратите внимание, что при работе с несколькими уровнями программы нам нужно получать результаты последовательно: запрос – ответ – запрос – ответ.

Точно так же происходит в диаграмме последовательности. Сама суть названия диаграммы говорит о том, что все должно происходить структурированно друг за другом.

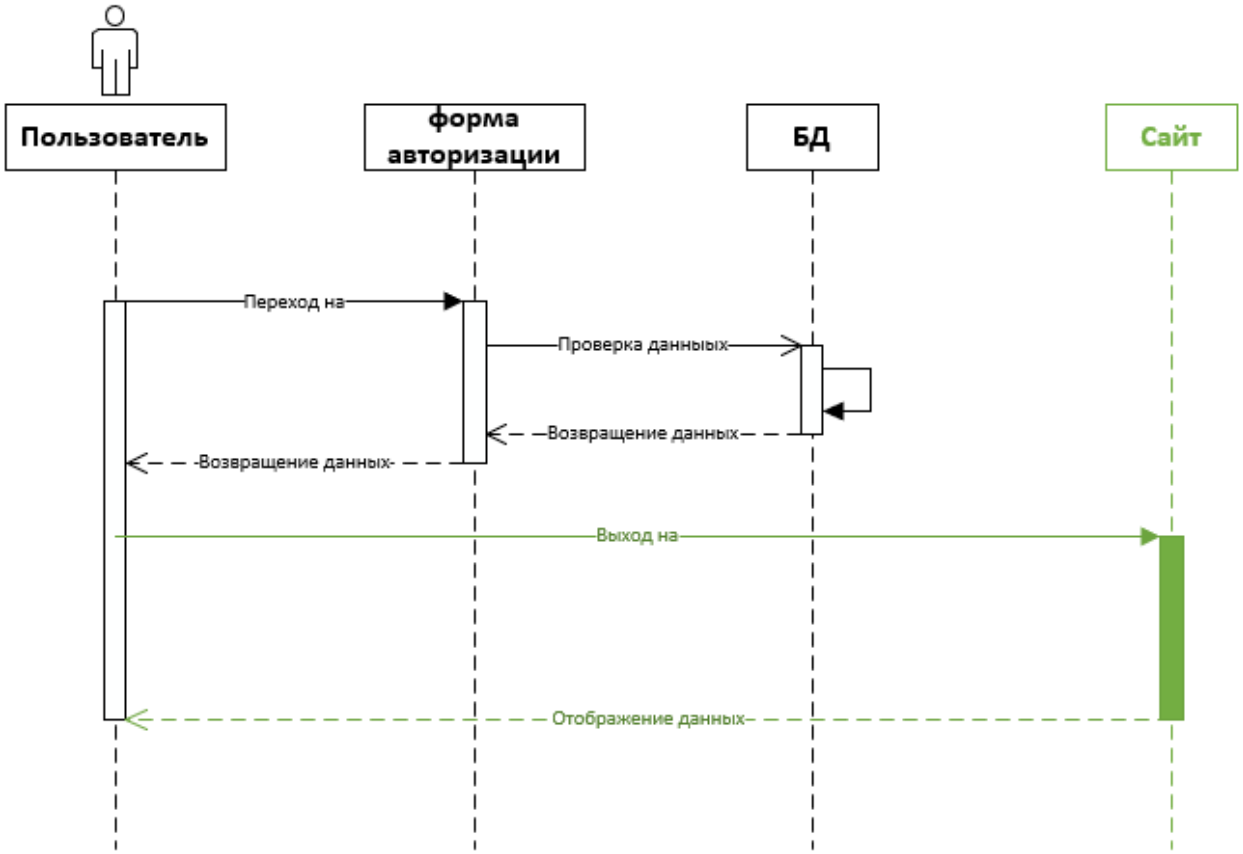


Рисунок 11 – Финальная обработка диаграммы.

Практическая работа №4.

Задание.

1. Возьмите вариант задания с практической работы № 2 и 3. Проработайте и доработайте диаграмму, чтобы она соответствовала новой теме.

2. Вспомнить работу с графическим редактором Draw.io для создания алгоритмов и диаграмм.